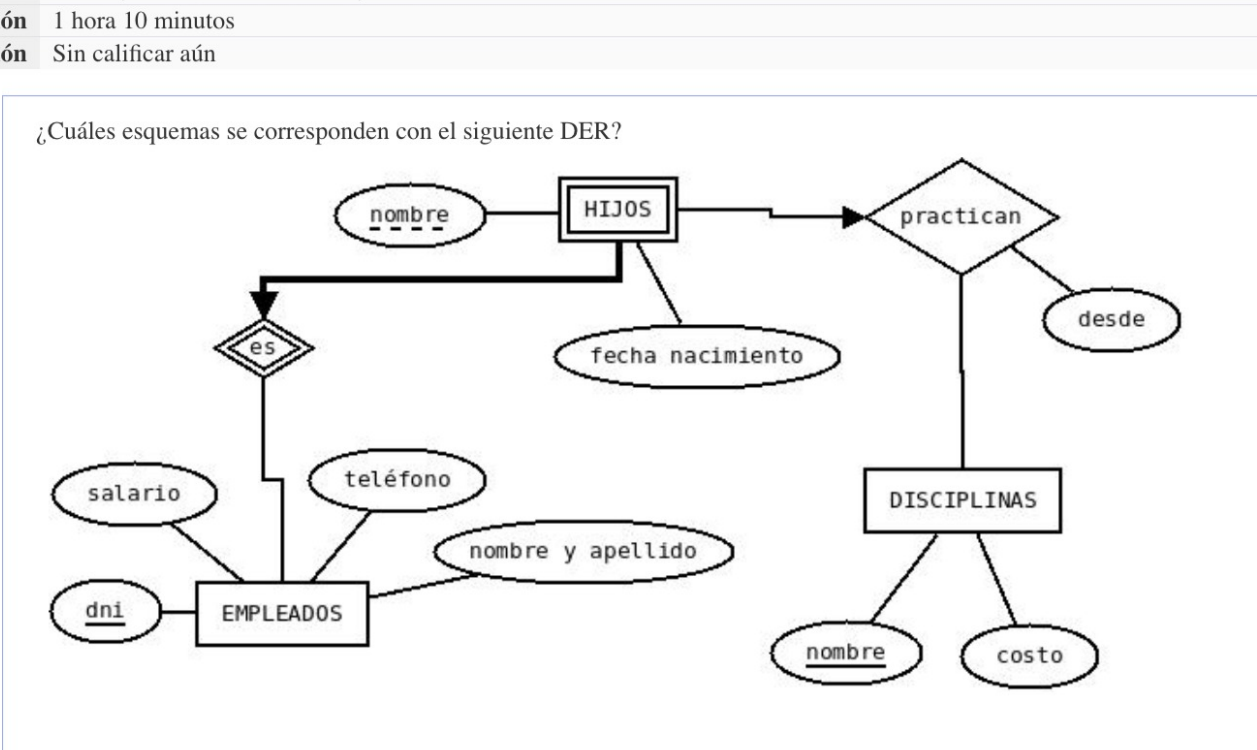
Estado	Finalizado
Comenzado	martes, 21 de octubre de 2025, 20:17
Completado	martes, 21 de octubre de 2025, 21:27
Duración	1 hora 10 minutos
Calificación	Sin calificar aún

Pregunta **1**
Correcta
Se puntúa 0,50 sobre 0,50
Marcar pregunta

¿Cuáles esquemas se corresponden con el siguiente DER?



Seleccione una:

○ a.
```
CREATE TABLE IF NOT EXISTS `EMPLEADOS` (
  `dni` INT(8) NOT NULL,
  `nombre` VARCHAR(50) NOT NULL,
  `salario` DECIMAL(5,2) NOT NULL,
  `telefono` VARCHAR(15) NULL,
  PRIMARY KEY (`dni`))
ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS `HIJOS` (
  `hijo_de` INT(8) NOT NULL,
  `nombre` VARCHAR(50) NOT NULL,
  `fecha_nacimiento` DATE NOT NULL,
  PRIMARY KEY (`hijo_de`, `nombre`),
  INDEX `fk_HIJOS_EMPLEADOS1_idx` (`hijo_de` ASC),
  CONSTRAINT `fk_HIJOS_EMPLEADOS1`
    FOREIGN KEY (`hijo_de`)
    REFERENCES `EMPLEADOS` (`dni`)
    ON DELETE CASCADE
    ON UPDATE CASCADE)
ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS `DISCIPLINAS` (
  `nombre` VARCHAR(20) NOT NULL,
  `costo` REAL NULL,
  PRIMARY KEY (`nombre`))
ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS `practican` (
  `empleado` INT(8) NOT NULL,
  `HIJOS_nombre` VARCHAR(50) NOT NULL,
  `DISCIPLINAS_nombre` VARCHAR(20) NOT NULL,
  `desde` DATE NOT NULL,
  PRIMARY KEY (`HIJOS_hijo_de`, `HIJOS_nombre`, `DISCIPLINAS_nombre`),
  INDEX `fk_HIJOS_DISCIPLINAS_DISCIPLINAS1_idx` (`DISCIPLINAS_nombre` ASC),
  INDEX `fk_HIJOS_DISCIPLINAS_HIJOS1_idx` (`HIJOS_nombre` ASC),
  INDEX `fk_EMPLEADOS1_idx` (`empleado` ASC),
  CONSTRAINT `fk_EMPLEADOS1`
    FOREIGN KEY (`empleado`)
    REFERENCES `EMPLEADOS` (`dni`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
  CONSTRAINT `fk_HIJOS_DISCIPLINAS_HIJOS1`
    FOREIGN KEY (`HIJOS_nombre`)
    REFERENCES `HIJOS` (`nombre`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
  CONSTRAINT `fk_HIJOS_DISCIPLINAS_DISCIPLINAS1`
    FOREIGN KEY (`DISCIPLINAS_nombre`)
    REFERENCES `DISCIPLINAS` (`nombre`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

○ b.
```
CREATE TABLE IF NOT EXISTS `EMPLEADOS` (
  `dni` INT(8) NOT NULL,
  `nombre` VARCHAR(50) NOT NULL,
  `salario` DECIMAL(5,2) NOT NULL,
  `telefono` VARCHAR(15) NULL,
  PRIMARY KEY (`dni`))
ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS `HIJOS` (
  `hijo_de` INT(8) NOT NULL,
  `nombre` VARCHAR(50) NOT NULL,
  `fecha_nacimiento` DATE NOT NULL,
  PRIMARY KEY (`hijo_de`, `nombre`),
  INDEX `fk_HIJOS_EMPLEADOS1_idx` (`hijo_de` ASC),
  CONSTRAINT `fk_HIJOS_EMPLEADOS1`
    FOREIGN KEY (`hijo_de`)
    REFERENCES `EMPLEADOS` (`dni`)
    ON DELETE CASCADE
    ON UPDATE CASCADE)
ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS `DISCIPLINAS` (
  `nombre` VARCHAR(20) NOT NULL,
  `costo` REAL NULL,
  PRIMARY KEY (`nombre`))
ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS `practican` (
  `HIJOS_nombre` VARCHAR(50) NOT NULL,
  `DISCIPLINAS_nombre` VARCHAR(20) NOT NULL,
  `desde` DATE NOT NULL,
  PRIMARY KEY (`HIJOS_nombre`, `DISCIPLINAS_nombre`),
  INDEX `fk_HIJOS_DISCIPLINAS_DISCIPLINAS1_idx` (`DISCIPLINAS_nombre` ASC),
  INDEX `fk_HIJOS_DISCIPLINAS_HIJOS1_idx` (`HIJOS_nombre` ASC),
  CONSTRAINT `fk_HIJOS_DISCIPLINAS_HIJOS1`
    FOREIGN KEY (`HIJOS_nombre`)
    REFERENCES `HIJOS` (`nombre`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
  CONSTRAINT `fk_HIJOS_DISCIPLINAS_DISCIPLINAS1`
    FOREIGN KEY (`DISCIPLINAS_nombre`)
    REFERENCES `DISCIPLINAS` (`nombre`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

● c. ✓
```
CREATE TABLE IF NOT EXISTS `EMPLEADOS` (
  `dni` INT(8) NOT NULL,
  `nombre` VARCHAR(50) NOT NULL,
  `salario` DECIMAL(5,2) NOT NULL,
  `telefono` VARCHAR(15) NULL,
  PRIMARY KEY (`dni`))
ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS `HIJOS` (
  `hijo_de` INT(8) NOT NULL,
  `nombre` VARCHAR(50) NOT NULL,
  `fecha_nacimiento` DATE NOT NULL,
  `DISCIPLINAS_nombre` VARCHAR(20),
  `desde` DATE NOT NULL,
  PRIMARY KEY (`hijo_de`, `nombre`),
  INDEX `fk_HIJOS_EMPLEADOS1_idx` (`hijo_de` ASC),
  INDEX `fk_HIJOS_DISCIPLINAS1_idx` (`DISCIPLINAS_nombre` ASC),
  CONSTRAINT `fk_HIJOS_EMPLEADOS1`
    FOREIGN KEY (`hijo_de`)
    REFERENCES `EMPLEADOS` (`dni`)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  CONSTRAINT `fk_HIJOS_DISCIPLINAS1`
    FOREIGN KEY (`DISCIPLINAS_nombre`)
    REFERENCES `DISCIPLINAS` (`nombre`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS `DISCIPLINAS` (
  `nombre` VARCHAR(20) NOT NULL,
  `costo` REAL NULL,
  PRIMARY KEY (`nombre`))
ENGINE = InnoDB;
```

○ d.
```
CREATE TABLE IF NOT EXISTS `EMPLEADOS` (
  `dni` INT(8) NOT NULL,
  `nombre` VARCHAR(50) NOT NULL,
  `salario` DECIMAL(5,2) NOT NULL,
  `telefono` VARCHAR(15) NULL,
  PRIMARY KEY (`dni`))
ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS `HIJOS` (
  `hijo_de` INT(8) NOT NULL,
  `nombre` VARCHAR(50) NOT NULL,
  `fecha_nacimiento` DATE NOT NULL,
  PRIMARY KEY (`hijo_de`, `nombre`),
  INDEX `fk_HIJOS_EMPLEADOS1_idx` (`hijo_de` ASC),
  CONSTRAINT `fk_HIJOS_EMPLEADOS1`
    FOREIGN KEY (`hijo_de`)
    REFERENCES `EMPLEADOS` (`dni`)
    ON DELETE CASCADE
    ON UPDATE CASCADE)
ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS `DISCIPLINAS` (
  `nombre` VARCHAR(20) NOT NULL,
  `costo` REAL NULL,
  PRIMARY KEY (`nombre`))
ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS `practican` (
  `HIJOS_hijo_de` INT(8) NOT NULL,
  `HIJOS_nombre` VARCHAR(50) NOT NULL,
  `DISCIPLINAS_nombre` VARCHAR(20) NOT NULL,
  `desde` DATE NOT NULL,
  PRIMARY KEY (`HIJOS_hijo_de`, `HIJOS_nombre`, `DISCIPLINAS_nombre`),
  INDEX `fk_HIJOS_DISCIPLINAS_DISCIPLINAS1_idx` (`DISCIPLINAS_nombre` ASC),
  INDEX `fk_HIJOS_DISCIPLINAS_HIJOS1_idx` (`HIJOS_hijo_de` ASC, `HIJOS_nombre` ASC),
  CONSTRAINT `fk_HIJOS_DISCIPLINAS_HIJOS1`
    FOREIGN KEY (`HIJOS_hijo_de` , `HIJOS_nombre`)
    REFERENCES `HIJOS` (`hijo_de` , `nombre`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
  CONSTRAINT `fk_HIJOS_DISCIPLINAS_DISCIPLINAS1`
    FOREIGN KEY (`DISCIPLINAS_nombre`)
    REFERENCES `DISCIPLINAS` (`nombre`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

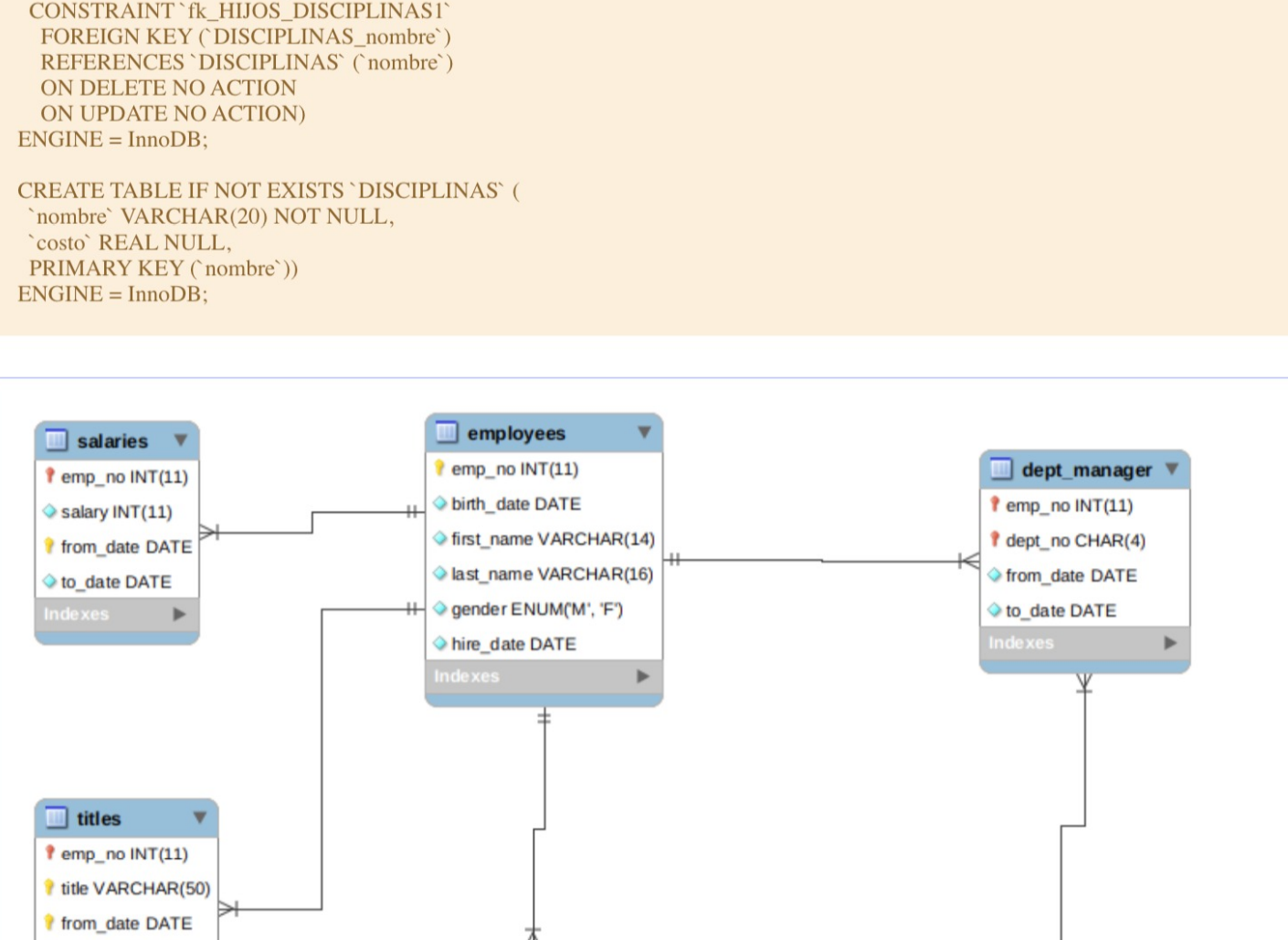
Respuesta correcta

La respuesta correcta es:
```
CREATE TABLE IF NOT EXISTS `EMPLEADOS` (
  `dni` INT(8) NOT NULL,
  `nombre` VARCHAR(50) NOT NULL,
  `salario` DECIMAL(5,2) NOT NULL,
  `telefono` VARCHAR(15) NULL,
  PRIMARY KEY (`dni`))
ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS `HIJOS` (
  `hijo_de` INT(8) NOT NULL,
  `nombre` VARCHAR(50) NOT NULL,
  `fecha_nacimiento` DATE NOT NULL,
  `DISCIPLINAS_nombre` VARCHAR(20),
  `desde` DATE NOT NULL,
  PRIMARY KEY (`hijo_de`, `nombre`),
  INDEX `fk_HIJOS_EMPLEADOS1_idx` (`hijo_de` ASC),
  INDEX `fk_HIJOS_DISCIPLINAS1_idx` (`DISCIPLINAS_nombre` ASC),
  CONSTRAINT `fk_HIJOS_EMPLEADOS1`
    FOREIGN KEY (`hijo_de`)
    REFERENCES `EMPLEADOS` (`dni`)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  CONSTRAINT `fk_HIJOS_DISCIPLINAS1`
    FOREIGN KEY (`DISCIPLINAS_nombre`)
    REFERENCES `DISCIPLINAS` (`nombre`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS `DISCIPLINAS` (
  `nombre` VARCHAR(20) NOT NULL,
  `costo` REAL NULL,
  PRIMARY KEY (`nombre`))
ENGINE = InnoDB;
```

Pregunta **2**
Parcialmente correcta
Se puntúa 0,67 sobre 1,00
Marcar pregunta



El esquema corresponde a la base de datos del subsistema de administración de una empresa multinacional con cerca de 300000 empleados. De los empleados (employees) se registra el historial de salarios (salaries) y de cargos (titles) que va teniendo a lo largo de su carrera en la empresa. Cada empleado trabaja (dept_emp) en uno o más departamentos (departments) registrándose el período, y cada departamento es administrado (dept_manager) por un empleado, pero los administradores pueden ir cambiando con el tiempo, registrándose los períodos en que cumplieron esa función.

Nota: para los casos en que los datos de un registro son actuales, las fechas de finalización de un período (to_date) se especifican como "9999-01-01", por lo tanto si hay algún cambio en el período actual (cambio de salario, cargo, lugar de trabajo, etc.) se debe actualizar ese campo a la fecha correspondiente.

SE PIDE:

i. El empleado Ramón Gutiérrez es designado administrador del departamento Contaduría. Esto a su vez implica un incremento del 15% en su salario.

ii. Crear una vista que muestre, para los departamentos con menos de 15000 empleados, el nombre de los departamentos y el nombre y apellido de los empleados que trabajan en ellos con sus cargos y sueldos actuales.

iii. Crear un índice que permita optimizar las consultas las fechas en que comenzó el trabajo de un empleado en un departamento.

Seleccionar las sentencias que satisfagan los requerimientos pedidos.

Seleccione una o más de una:

☐ a.
```
INSERT INTO dept_emp
SELECT e.emp_no, d.dept_no, curdate(), '9999-01-01'
   FROM employees e, departments d
   WHERE e.first_name = 'Ramón' AND e.last_name = 'Gutiérrez' AND d.dept_name = 'Contaduría';
UPDATE titles t
SET t.title = 'Manager'
UPDATE salaries s, employees e
SET s.salary = s.salary * 1.15
   WHERE e.first_name = 'Ramón' AND e.last_name = 'Gutiérrez';
```

☑ b. ✓
```
CREATE INDEX fecha_comienzo ON dept_emp (emp_no, from_date);
```

☐ c.
```
CREATE VIEW detalle_deptos AS (
SELECT d.dept_name, e.first_name, e.last_name
FROM departments d INNER JOIN dept_emp de ON d.dept_no = de.dept_no
INNER JOIN employees e ON de.emp_no = e.emp_no,
LATERAL (SELECT COUNT(*) AS cant
FROM dept_emp de1
      WHERE de1.dept_no = d.dept_no AND de1.to_date = '9999-01-01') AS tot_depto
WHERE de.to_date = '9999-01-01' AND tot_depto.cant < 15000);
```

☐ d.
```
UPDATE dept_emp
ADD INDEX (emp_no, from_date);
```

☑ e. ✓
```
UPDATE dept_manager dm
SET dm.to_date = curdate()
   WHERE dm.to_date = '9999-01-01' AND dm.dept_no = (
SELECT d.dept_no
      FROM departments d
      WHERE d.dept_name = 'Contaduría'
   );
INSERT INTO dept_manager
SELECT e.emp_no, d.dept_no, curdate(), '9999-01-01'
   FROM employees e, departments d
   WHERE e.first_name = 'Ramón' AND e.last_name = 'Gutiérrez' AND d.dept_name = 'Contaduría';
UPDATE salaries s
SET s.to_date = curdate()
   WHERE s.to_date = '9999-01-01' AND s.emp_no = (
SELECT e.emp_no
      FROM employees e
      WHERE e.first_name = 'Ramón' AND e.last_name = 'Gutiérrez'
   );
INSERT INTO salaries
SELECT e.emp_no, s.salary * 1.15, curdate(), '9999-01-01'
   FROM employees e INNER JOIN salaries s ON e.emp_no = s.emp_no
   WHERE e.first_name = 'Ramón' AND e.last_name = 'Gutiérrez' AND s.to_date = curdate();
```

Respuesta parcialmente correcta.

Ha seleccionado correctamente 2.

```
/*
* El empleado Ramón Gutiérrez es designado administrador del departamento Contaduría. Esto a su vez implica un incremento del 15% en su salario.
*/
UPDATE dept_manager dm
SET dm.to_date = curdate()
   WHERE dm.to_date = '9999-01-01' AND dm.dept_no = (
SELECT d.dept_no
      FROM departments d
      WHERE d.dept_name = 'Contaduría'
   );
INSERT INTO dept_manager
SELECT e.emp_no, d.dept_no, curdate(), '9999-01-01'
   FROM employees e, departments d
   WHERE e.first_name = 'Ramón' AND e.last_name = 'Gutiérrez' AND d.dept_name = 'Contaduría';
UPDATE salaries s
SET s.to_date = curdate()
   WHERE s.to_date = '9999-01-01' AND s.emp_no = (
SELECT e.emp_no
      FROM employees e
      WHERE e.first_name = 'Ramón' AND e.last_name = 'Gutiérrez'
   );
INSERT INTO salaries
SELECT e.emp_no, s.salary * 1.15, curdate(), '9999-01-01'
   FROM employees e INNER JOIN salaries s ON e.emp_no = s.emp_no
   WHERE e.first_name = 'Ramón' AND e.last_name = 'Gutiérrez' AND s.to_date = curdate();
/*
* Crear una vista que muestre, para los departamentos con menos de 15000 empleados,
el nombre de los departamentos y el nombre y apellido de los empleados que trabajan en ellos.
*/
CREATE VIEW detalle_deptos AS (
SELECT d.dept_name, e.first_name, e.last_name
FROM departments d INNER JOIN dept_emp de ON d.dept_no = de.dept_no
INNER JOIN employees e ON de.emp_no = e.emp_no,
LATERAL (SELECT COUNT(*) AS cant
FROM dept_emp de1
      WHERE de1.dept_no = d.dept_no AND de1.to_date = '9999-01-01') AS tot_depto
WHERE de.to_date = '9999-01-01' AND tot_depto.cant < 15000);
/*
* Crear un índice que permita optimizar las consultas las fechas en que comenzó el trabajo de un empleado en un departamento.
*/
CREATE INDEX fecha_comienzo ON dept_emp (emp_no, from_date);
```

Las respuestas correctas son:
```
UPDATE dept_manager dm
SET dm.to_date = curdate()
   WHERE dm.to_date = '9999-01-01' AND dm.dept_no = (
SELECT d.dept_no
      FROM departments d
      WHERE d.dept_name = 'Contaduría'
   );
INSERT INTO dept_manager
SELECT e.emp_no, d.dept_no, curdate(), '9999-01-01'
   FROM employees e, departments d
   WHERE e.first_name = 'Ramón' AND e.last_name = 'Gutiérrez' AND d.dept_name = 'Contaduría';
UPDATE salaries s
SET s.to_date = curdate()
   WHERE s.to_date = '9999-01-01' AND s.emp_no = (
SELECT e.emp_no
      FROM employees e
      WHERE e.first_name = 'Ramón' AND e.last_name = 'Gutiérrez'
   );
INSERT INTO salaries
SELECT e.emp_no, s.salary * 1.15, curdate(), '9999-01-01'
   FROM employees e INNER JOIN salaries s ON e.emp_no = s.emp_no
   WHERE e.first_name = 'Ramón' AND e.last_name = 'Gutiérrez' AND s.to_date = curdate();
```
,
```
CREATE VIEW detalle_deptos AS (
SELECT d.dept_name, e.first_name, e.last_name
FROM departments d INNER JOIN dept_emp de ON d.dept_no = de.dept_no
INNER JOIN employees e ON de.emp_no = e.emp_no,
LATERAL (SELECT COUNT(*) AS cant
FROM dept_emp de1
      WHERE de1.dept_no = d.dept_no AND de1.to_date = '9999-01-01') AS tot_depto
WHERE de.to_date = '9999-01-01' AND tot_depto.cant < 15000);
```
,
```
CREATE INDEX fecha_comienzo ON dept_emp (emp_no, from_date);
```

Pregunta **3**
Correcta
Se puntúa 0,50 sobre 0,50
Marcar pregunta

Dado el esquema de la siguiente tabla:

Empleados (nro_emp, fecha_nacimiento, nombres, apellidos, fecha_contrato, sueldo, cod_depto)

¿A qué consigna responde la siguiente consulta?

```
SELECT e.apellidos, e.nombres
FROM Empleados e
WHERE e.cod_depto IN (SELECT e1.cod_depto
                      FROM Empleados e1
                      WHERE e1.sueldo = (SELECT MAX(e2.sueldo)
                                         FROM Empleados e2))
```

Seleccione una:

○ a. Listar los apellidos y nombres con el mayor sueldo de cada departamento.
○ b. Listar los apellidos y nombres de los empleados con el mayor sueldo.
● c. Encontrar los empleados que trabajan en los departamentos donde se encuentran los mayores sueldos. ✓
○ d. La consulta está mal planteada y da error.

Respuesta correcta

La respuesta correcta es:
Encontrar los empleados que trabajan en los departamentos donde se encuentran los mayores sueldos.

Pregunta **4**
Finalizado
Se puntúa como 0 sobre 1,00
Marcar pregunta

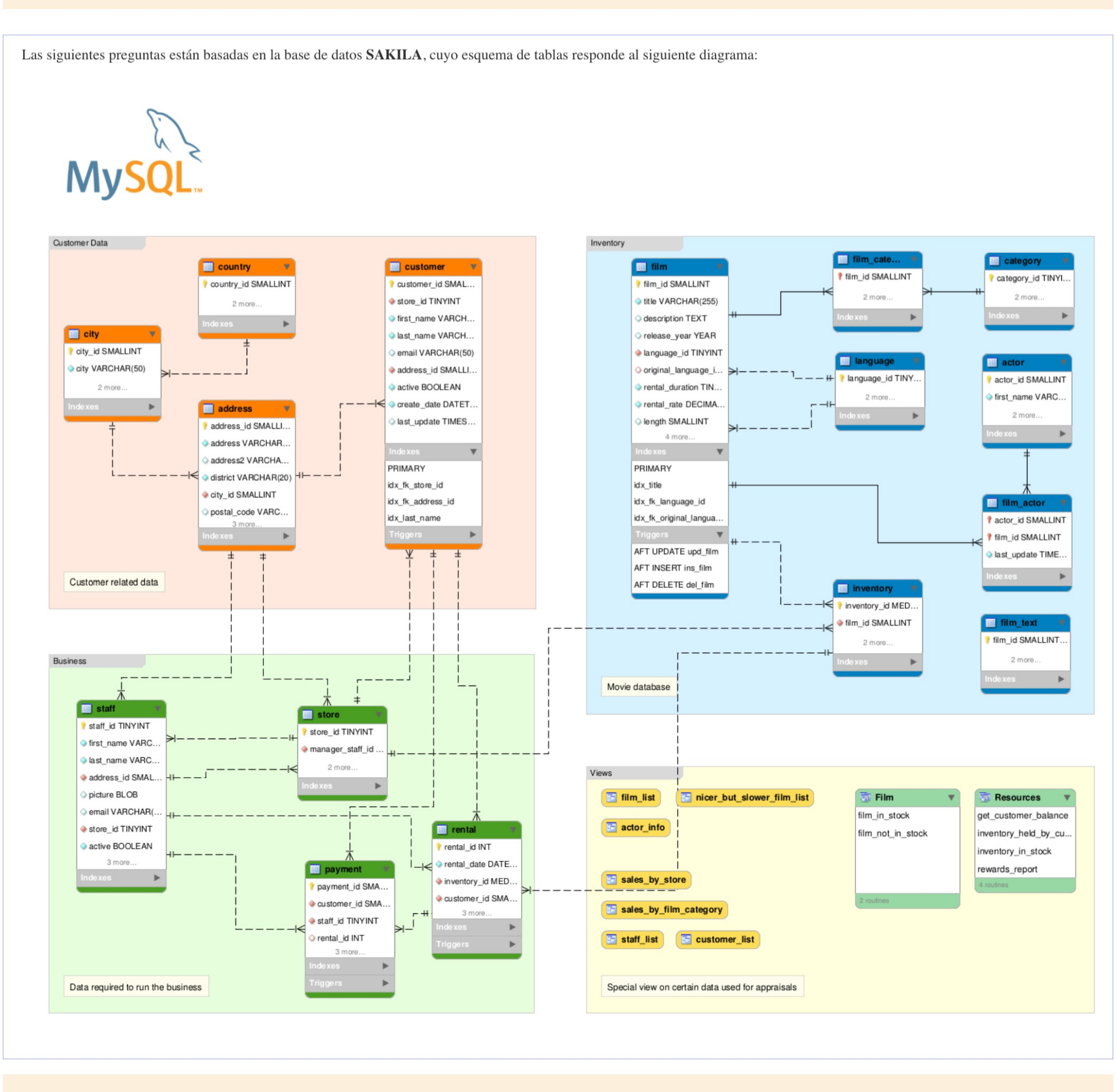
Dado el esquema de la BD de un sistema de encomiendas:

Cliente (id_cliente, nombre_cliente, tipo_cliente, fecha_alta)
Embarque (num_embarque, id_cliente, peso, destino, fecha_embarque)
id_cliente → Cliente; camión → Camión; destino → Ciudad
Camión (num_camión, nombre_chofer)
Ciudad (id_ciudad, nom_ciudad, población)

Elabore una consulta que responda a la consigna "*Encontrar los nombres de los choferes que llevaron embarques a todas las ciudades registradas*".

Información
Marcar pregunta

Las siguientes preguntas están basadas en la base de datos **SAKILA**, cuyo esquema de tablas responde al siguiente diagrama:



Pregunta **5**
Parcialmente correcta
Se puntúa 0,67 sobre 1,00
Desmarcar

Qué consulta/s SQL responde/n adecuadamente a la pregunta ***¿A qué categoría corresponden la mayoría de las películas?***

☑ a.
```
SELECT c.name
FROM category c INNER JOIN (
      SELECT fc.category_id, count(*) AS cant
      FROM film_category fc
   GROUP BY fc.category_id) AS aux1 ON c.category_id = aux1.category_id
WHERE aux1.cant >= ALL (
         SELECT count(*) AS cant
         FROM film_category fc
         GROUP BY fc.category_id
);
```

☐ b.
```
SELECT c.name
FROM category c INNER JOIN (
      SELECT fc.category_id, count(*) AS cant
      FROM film_category fc
   GROUP BY fc.category_id) AS aux1 ON c.category_id = aux1.category_id
WHERE aux1.cant = (
      SELECT MAX(aux2.cant)
   FROM (
         SELECT fc.category_id, count(*) AS cant
         FROM film_category fc
         GROUP BY fc.category_id) AS aux2
);
```

☑ c. ✓
```
WITH aux1 AS (
      SELECT fc.category_id, count(*) AS cant
      FROM film_category fc
   GROUP BY fc.category_id)

SELECT c.name
FROM category c, aux1
WHERE c.category_id = aux1.category_id AND aux1.cant = (
      SELECT MAX(aux1.cant)
   FROM aux1
);
```

☐ d. Ninguna de las alternativas responde adecuadamente a la pregunta.

Respuesta parcialmente correcta.

Ha seleccionado correctamente 2.

Las respuestas correctas son:
```
WITH aux1 AS (
      SELECT fc.category_id, count(*) AS cant
      FROM film_category fc
   GROUP BY fc.category_id)

SELECT c.name
FROM category c, aux1
WHERE c.category_id = aux1.category_id AND aux1.cant = (
      SELECT MAX(aux1.cant)
   FROM aux1
);,
SELECT c.name
FROM category c INNER JOIN (
      SELECT fc.category_id, count(*) AS cant
      FROM film_category fc
   GROUP BY fc.category_id) AS aux1 ON c.category_id = aux1.category_id
WHERE aux1.cant = (
      SELECT MAX(aux2.cant)
   FROM (
         SELECT fc.category_id, count(*) AS cant
         FROM film_category fc
         GROUP BY fc.category_id) AS aux2
);,
SELECT c.name
FROM category c INNER JOIN (
      SELECT fc.category_id, count(*) AS cant
      FROM film_category fc
   GROUP BY fc.category_id) AS aux1 ON c.category_id = aux1.category_id
WHERE aux1.cant >= ALL (
         SELECT count(*) AS cant
         FROM film_category fc
         GROUP BY fc.category_id
);
```

Pregunta **6**
Correcta
Se puntúa 1,00 sobre 1,00
Desmarcar

Qué consulta/s SQL responde/n adecuadamente a la pregunta ***¿Qué cliente alquiló la película LAWLESS VISION el 24/05/2005?***

○ a. Ninguna de las alternativas responde adecuadamente a la pregunta.

● b. ✓
```
SELECT aux2.first_name, aux2.last_name
FROM film f, LATERAL (
      SELECT r.*
   FROM rental r INNER JOIN inventory i ON r.inventory_id = i.inventory_id
   WHERE f.film_id = i.film_id) AS aux1, LATERAL (
   SELECT c.*
   FROM customer c
   WHERE c.customer_id = aux1.customer_id) AS aux2
WHERE f.title = 'LAWLESS VISION' AND DATE(aux1.rental_date) = '2005-05-24';
```

○ c. Ambas alternativas responde adecuadamente a la pregunta.

○ d.
```
SELECT c.first_name, c.last_name
FROM film f, customer c, (
      SELECT r.*
   FROM rental r INNER JOIN inventory i ON r.inventory_id = i.inventory_id) AS aux1
WHERE c.customer_id = aux1.customer_id AND f.title LIKE 'LAWLESS VISION' AND aux1.rental_date = '2005-05-24';
```

Respuesta correcta

La respuesta correcta es:
```
SELECT aux2.first_name, aux2.last_name
FROM film f, LATERAL (
      SELECT r.*
   FROM rental r INNER JOIN inventory i ON r.inventory_id = i.inventory_id
   WHERE f.film_id = i.film_id) AS aux1, LATERAL (
   SELECT c.*
   FROM customer c
   WHERE c.customer_id = aux1.customer_id) AS aux2
WHERE f.title = 'LAWLESS VISION' AND DATE(aux1.rental_date) = '2005-05-24';
```

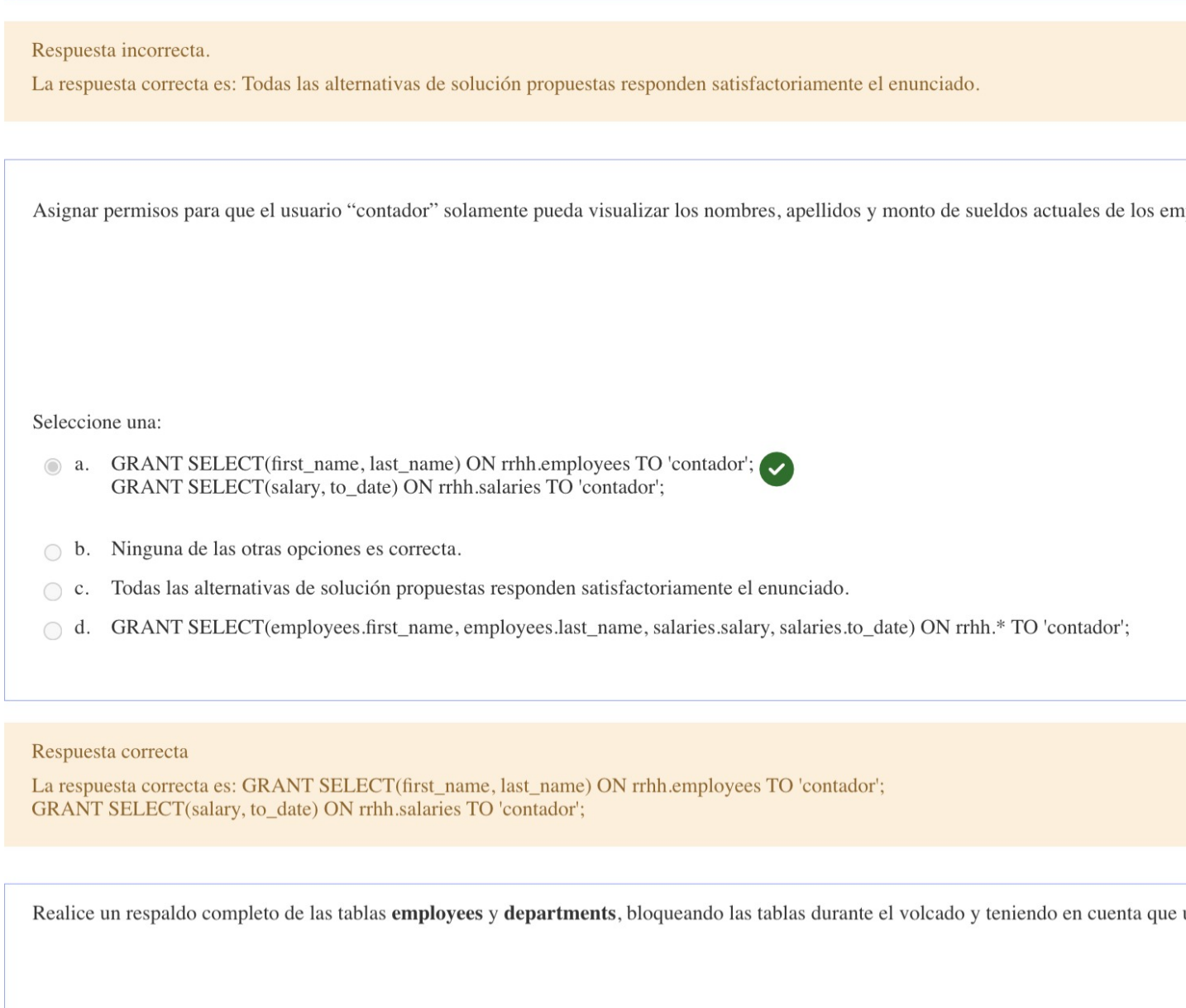
Información
Marcar pregunta

Enunciado común para los siguientes ejercicios prácticos.

El siguiente diagrama corresponde al esquema de la base de datos **"rrhh"** del subsistema de administración de recursos humanos de una empresa multinacional con cerca de 300000 empleados.

De los empleados (**employees**), además de los datos básicos (*número de empleado, fecha de nacimiento, nombre, apellido, género, fecha de contrato*), se registra el historial de salarios (**salaries**) y de cargos (**titles**) que van teniendo a lo largo de su carrera en la empresa. Cada empleado puede trabajar (**dept_emp**) en uno o más departamentos (**departments**) registrándose el período, y cada departamento es administrado (**dept_manager**) por un empleado, pero los administradores pueden ir cambiando con el tiempo, registrándose los períodos en que cumplieron esa función.

Nota: para los casos en que los datos de un **registro** son **actuales**, las fechas de finalización de un período (**to_date**) se especifican como **"9999-01-01"**, por lo tanto si hay algún cambio en el período actual (cambio de salario, cargo, lugar de trabajo, gerencia de un departamento) se debe actualizar ese campo a la fecha correspondiente.



Los ejercicios consisten en **dos tipos de preguntas: opción múltiple y ensayos** (preguntas abiertas). **Elija la/s opción/es** que considere adecuada/s **o escriba las sentencias/comandos** necesarios para cumplir con el enunciado **según el caso**.

Pregunta **7**
Incorrecta
Se puntúa 0,00 sobre 1,00
Desmarcar

Crear una rutina almacenada que, dado un número de empleado verifique en la tabla **dept_manager** si actualmente es un gerente y devuelva la cantidad de empleado que trabajan actualmente en el departamento a su cargo.

Seleccione una:

○ a. Todas las alternativas de solución propuestas responden satisfactoriamente el enunciado.

○ b. Ninguna de las otras opciones es correcta.

○ c.
```
CREATE FUNCTION personal_a_cargo(gerente int)
RETURNS int
BEGIN
   DECLARE empleados int;
   SET empleados = (SELECT COUNT(*)
                    FROM dept_manager dm INNER JOIN dept_emp de ON dm.dept_no = de.dept_no
                    WHERE dm.emp_no = gerente AND dm.to_date = '9999-01-01' AND de.to_date = '9999-01-01');
   RETURN empleados;
END//
```

● d. ✗
```
CREATE PROCEDURE personal_a_cargo(gerente int)
BEGIN
   SET @empleados = (SELECT COUNT(*)
                     FROM dept_manager dm INNER JOIN dept_emp de ON dm.dept_no = de.dept_no
                     WHERE dm.emp_no = gerente AND dm.to_date = '9999-01-01' AND de.to_date = '9999-01-01');
   SELECT @empleados;
END//
```

Respuesta incorrecta.

La respuesta correcta es: Todas las alternativas de solución propuestas responden satisfactoriamente el enunciado.

Pregunta **8**
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Asignar permisos para que el usuario "contador" solamente pueda visualizar los nombres, apellidos y monto de sueldos actuales de los empleados.

Seleccione una:

● a. ✓
```
GRANT SELECT(first_name, last_name) ON rrhh.employees TO 'contador';
GRANT SELECT(salary, to_date) ON rrhh.salaries TO 'contador';
```
○ b. Ninguna de las otras opciones es correcta.
○ c. Todas las alternativas de solución propuestas responden satisfactoriamente el enunciado.
○ d. `GRANT SELECT(employees.first_name, employees.last_name, salaries.salary, salaries.to_date) ON rrhh.* TO 'contador';`

Respuesta correcta

La respuesta correcta es: GRANT SELECT(first_name, last_name) ON rrhh.employees TO 'contador';
GRANT SELECT(salary, to_date) ON rrhh.salaries TO 'contador';

Pregunta **9**
Incorrecta
Se puntúa 0,00 sobre 1,00
Marcar pregunta

Realice un respaldo completo de las tablas **employees** y **departments**, bloqueando las tablas durante el volcado y teniendo en cuenta que utilizan el engine InnoDB.

Seleccione una:

○ a.
```
SELECT *
FROM employees, departments
INTO DUMPFILE 'respaldo_completo.sql';
```
○ b. `mysqldump -u user_backup -p --single-transaction rrhh employees departments > respaldo_completo.sql`
● c. Ninguna de las otras opciones es correcta. ✗
○ d. Todas las alternativas de solución propuestas responden satisfactoriamente el enunciado.

Respuesta incorrecta.

La respuesta correcta es: mysqldump -u user_backup -p --single-transaction rrhh employees departments > respaldo_completo.sql

Pregunta **10**
Correcta
Se puntúa 0,50 sobre 0,50
Marcar pregunta

Para las tablas InnoDB en mysqldump se puede realizar una copia de seguridad en línea y que no produzca bloqueos en las tablas mediante el parametro ______________

Seleccione una:

● a. --single-transaction ✓
○ b. --double-transaction
○ c. --multiple-transaction
○ d. --no-transaction

La respuesta correcta es: --single-transaction

Pregunta **11**
Correcta
Se puntúa 0,50 sobre 0,50
Marcar pregunta

En MySQL, para obtener el instante actual (fecha y hora) , se emplea la función

Seleccione una:

○ a. Las anteriores respuestas no son correctas
○ b. CURDATE()
● c. NOW() ✓
○ d. TIME()

La respuesta correcta es: NOW()

Pregunta **12**
Correcta
Se puntúa 0,50 sobre 0,50
Marcar pregunta

¿Para cuál de los siguientes objetos no son compatibles los disparadores?

Seleccione una:

● a. views ✓
○ b. delete
○ c. update
○ d. insert

La respuesta correcta es: views

Pregunta **13**
Correcta
Se puntúa 0,50 sobre 0,50
Marcar pregunta

Con SQL, ¿cómo inserta un nuevo registro en la tabla "Persons"?

Seleccione una:

○ a. Ninguna es correcta
○ b. INSERT ('Jimmy', 'Jackson') INTO Persons
○ c. INSERT VALUES ('Jimmy', 'Jackson') INTO Persons
● d. INSERT INTO Persons VALUES ('Jimmy', 'Jackson') ✓

La respuesta correcta es: INSERT INTO Persons VALUES ('Jimmy', 'Jackson')

Finalizar revisión